

บทที่ 3

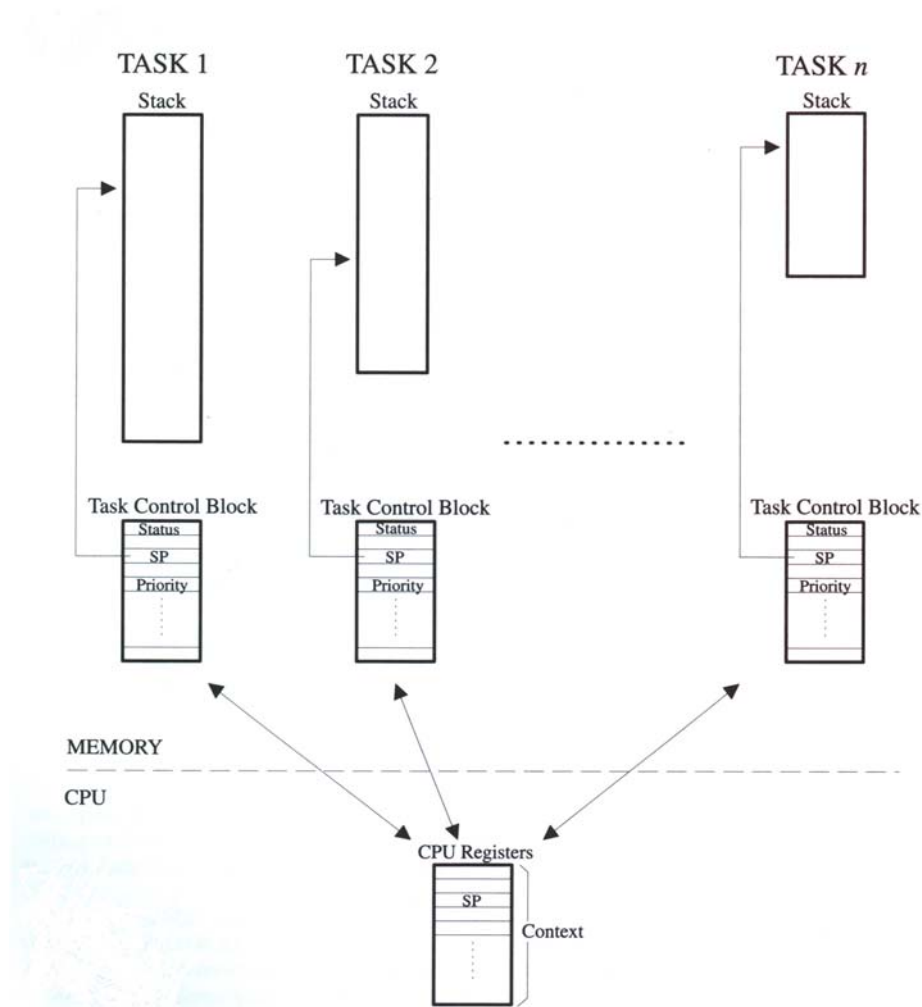
ทฤษฎีและหลักการของเทคนิคที่เกี่ยวข้อง

3.1 การเขียนโปรแกรมให้ทำงานเป็นมัลติทาสก์

เป็นโหมมการทำงานที่อนุญาตให้สามารถทำงานพร้อมๆกันได้ หรือ ประมวลผลแบบสอดแทรกกันของสองทาสก์ หรือมากกว่า คล้ายกับมัลติโปรแกรมมิ่งที่มีรูปแบบแตกต่างกัน

ในระบบปฏิบัติการคอมพิวเตอร์คือไม่อนุญาตโปรแกรมหลายๆ โปรแกรมทำงานพร้อมๆกันต้องทำงานให้เสร็จทีละงานเป็นลำดับไป ดังนั้นงานในการจัดการให้แต่ละ โปรแกรมหรือหลายๆงานทำไปพร้อมกันจึงเป็นของผู้สร้างโปรแกรม

ในปัจจุบันมีไลบรารีที่มาช่วยจัดการให้โปรแกรมหนึ่งๆ สามารถรันหลายๆฟังก์ชันสลับกันไปมาได้หลายตัวด้วยกัน อาทิเช่น ไมโครซีไอเอส (อ่านว่า MicroC/OS) , Multi-C library เป็นต้น ซึ่งในโครงการนี้ผู้พัฒนาได้เลือกใช้ uC/OS-II เนื่องจากไม่ขึ้นกับหน่วยประมวลผล และสนับสนุนการทำงานแบบเรียลไทม์ (real time)



รูปที่ 3-1 มัลติทาสก์กึ่ง

3.2 ไมโครคอนโทรลเลอร์ โอเพอเรตติ้ง ซิสเต็มเวอร์ชัน 2

ไมโครคอนโทรลเลอร์โอเพอเรตติ้งซิสเต็มเวอร์ชัน 2 หรือไมโครซีโอเอส2 (μC/OS-II) ถูกสร้างขึ้นบนพื้นฐานของเรียลไทม์เคอร์เนล (Real-time kernel) ซึ่งนิยมนำไปใช้ในอุปกรณ์หลายๆอย่างด้วยกัน เช่น กล้องถ่ายรูป เน็ตเวิร์คอะแดปเตอร์ อุตสาหกรรม เป็นต้น

3.2.1 จุดเด่นของไมโครคอนโทรลเลอร์ โอเพอเรตติ้ง ซิสเต็มเวอร์ชัน 2

1. ซอร์สโค้ด (Source Code)

มีการจัดระเบียบไว้เป็นอย่างดีและเข้าใจง่าย

2. พอร์แทเบิล (Portable)

โดยส่วนใหญ่แล้วไมโครซีโอเอส2 จะถูกเขียนโดย ANSI C ที่มีความสามารถในการพอร์แทเบิลสูง ด้วยการเขียนด้วยแอสเอ็ม (ASM) ที่ทำให้ไมโครซีโอเอส2 มีขนาดเล็กและสามารถพอร์ตไปยังชิพประมวลผลตัวอื่น ๆ ได้เช่นเดียวกันกับ ไมโครซีโอเอส ไมโครซีโอเอส2สามารถถูกพอร์ตลงไปในชิพประมวลผลตัวที่ใหญ่กว่าได้ คราบไคที่ชิพประมวลผลนั้นๆ มีแอสต็กพอยเตอร์ (stack pointer) และซีพียู (CPU register) ที่สามารถถูกพุช (push) และป๊อป (pop) ลงไปในแอสต็กได้

พอร์ตทั้งหมดที่มีอยู่ในไมโครซีโอเอสสามารถถูกแปลงมาลงในไมโครซีโอเอส2 ภายในเวลาประมาณ 1 ชม. เพราะไมโครซีโอเอส2 สามารถเข้ากันได้กับไมโครซีโอเอส ทำให้โปรแกรมประยุกต์ของไมโครซีโอเอสสามารถรันบนไมโครซีโอเอส2 ได้ด้วยการเปลี่ยนแปลงที่เล็กน้อยหรือไม่เปลี่ยนแปลงเลย สามารถตรวจสอบพอร์ตที่มีอยู่ในปัจจุบันของไมโครซีโอเอส2 ได้จาก web site www.uCOS-II.com

3. รอมเมเบิล (ROMable)

ไมโครซีโอเอส2ได้ถูกออกแบบสำหรับโปรแกรมประยุกต์ฝังตัวหมายความว่าถ้าเรามี proper tool chain (เช่น C compiler, assembler, และ linker/locator) คุณสามารถฝัง uC/OS-II ลงไปในส่วนของชิปงานของคุณได้

4. แสเกลเอเบิล (Scalable)

ผู้สร้างได้ออกแบบไมโครซีโอเอส2ในแบบที่เราสามารถใช้เฉพาะบริการที่เราต้องการในงานได้ หมายถึงงานของเราสามารถใช้งานบริการบนไมโครซีโอเอส2บางส่วน จนถึงใช้งานบริการทุกส่วน และยอมให้ลดขนาดของหน่วยความจำ (ทั้ง ROM และ RAM) ที่ถูกใช้งานโดยไมโครซีโอเอส2 บนแต่ละงานได้

5. 프리เอมทิฟ (Preemptive)

ไมโครซีโอเอส2เป็น “fully preemptive real-time kernel” ซึ่งหมายความว่า ไมโครซีโอเอส2จะทำงานที่มีความสำคัญสูงที่สุดที่พร้อมทำงานก่อนเสมอ ซึ่งโดยเคอร์เนลตามท้องตลาดส่วนใหญ่แล้วจะเป็นแบบฟรีเอมทิฟและไมโครซีโอเอส2เทียบเคียงได้กับเคอร์เนลเหล่านั้น

6. มัลติทาสกิ้ง (Multitasking)

ไมโครซีโอเอส2สามารถจัดการทาสก์ได้ถึง 64 ทาสก์พร้อมกัน อย่างไรก็ตามซอฟต์แวร์รุ่นปัจจุบันได้จองทาสก์ไว้ 8 ทาสก์เพื่อให้สำหรับระบบใช้งาน ทำให้เหลือเพียง 56 ทาสก์เท่านั้น แต่ละทาสก์จะมีเลขกำกับ

ที่ไม่ซ้ำกัน ซึ่งหมายความว่าไมโครซีไอเอส2ไม่สามารถที่จะทำ round-robin scheduling ได้เหตุนี้ทำให้มี 64 ระดับความสำคัญ (priority level)

7. ดีเทอร์มินิสติก (Deterministic)

เวลาในการประมวลผลของไมโครซีไอเอส2สามารถวัดได้ นั่นหมายถึงเราสามารถวัด รู้และคำนวณ เวลาที่ไมโครซีไอเอส2ใช้ในการคำนวณฟังก์ชันและบริการ แต่อย่างไรก็ตามเวลาในการประมวลผลของไมโครซีไอเอส2เซอร์วิสทั้งหมดไม่ขึ้นอยู่กับจำนวนของทาสก์ในงานของเรา

8. ทาสก์สแต็ก (Task Stacks)

แต่ละทาสก์ต้องการสแต็กของตัวเอง อย่างไรก็ตามไมโครซีไอเอส2ยอมให้แต่ละทาสก์มีขนาดของสแต็กที่แตกต่างกันได้ ซึ่งสามารถช่วยให้เราลดขนาดการใช้งานบนแรมบนงานของเราได้ด้วย stack-checking ของไมโครซีไอเอส2 เราสามารถประมาณค่าของสแต็กที่เราต้องการใช้บนแต่ละทาสก์ของเราได้อย่างเท่าที่เรากำลังใช้งานจริง ๆ

9. บริการ (Services)

ไมโครซีไอเอส2 ได้เตรียมบริการต่าง ๆ เช่น mailboxes queues semaphores partition เป็นต้น

10. การจัดการอินเทอร์รัพท์ (Interrupt Management)

อินเทอร์รัพท์สามารถหยุดการทำงานของทาสก์ไว้ชั่วคราวได้ โดยที่ทาสก์ที่มีความสำคัญที่สูงกว่าจะสามารถตื่นขึ้นมาทำงานระหว่างอินเทอร์รัพท์ของทาสก์ที่มีความสำคัญต่ำกว่าได้ โดยที่อินเทอร์รัพท์สามารถถูกใช้ได้ถึง 255 ระดับซ้อนกัน

11. มีความสามารถมากและเชื่อถือได้ (Robust and Reliable) ไมโครซีไอเอส2มีพื้นฐานอยู่บน ไมโครซีไอเอส ซึ่งถูกใช้ในหลายร้อยงานทั่วโลกมาตั้งแต่ปี 1992 ซึ่งไมโครซีไอเอส2ใช้คอร์ (core) เดิมและฟังก์ชันที่ยังคงอยู่เหมือนเดิมเป็นส่วนใหญ่

3.2.2 แนวคิดแบบเรียลไทม์ซิสเต็ม

เรียลไทม์ซิสเต็มได้ถูกกำหนดลักษณะโดยผลที่ตามมาซึ่งถูกบังคับอันเนื่องมาจากทางตรรกะ เช่นเดียวกันกับคุณสมบัติความถูกต้องของเวลาของระบบที่ไม่ได้เป็นดั่งนั้น แบ่งออกเป็น 2 ชนิดคือ ซอฟท์ (SOFT) และฮาร์ด (HARD) สำหรับ SOFT real-time system นั้นทาสก์จะถูกกระทำโดยระบบเร็วที่สุดที่ทราบเท่าที่จะทำได้ แต่ทาสก์อาจจะไม่เสร็จได้ทันตามเวลาที่กำหนดไว้ สำหรับแบบ Hard real-time system จะถูกกำหนดให้งานออกมาถูกต้องและเสร็จตามเวลาที่กำหนด เรียลไทม์ซิสเต็มโดยส่วนใหญ่แล้ว จะประกอบไปด้วยทั้งแบบ SOFT และ HARD real-time application นั้นมีกรอบคลุมกว้างมาก แต่เรียลไทม์ซิสเต็มส่วนใหญ่นั้นจะเป็นแบบฝังตัว หมายความว่าคอมพิวเตอร์ที่ถูกสร้างในระบบจะไม่ถูกผู้ใช้เห็น โดยที่เรียลไทม์ซิสเต็มนั้นออกแบบได้ยากกว่าระบบที่ไม่เรียลไทม์ซิสเต็มมาก

3.2.3 มิวเชอร์ลเอ็กซ์คลูชัน (Mutual Exclusion)

วิธีการที่ง่ายที่สุดสำหรับทาสก์ที่จะติดต่อสื่อสารกันผ่านข้อมูลที่ใช้ร่วมกัน เมื่อมีการแลกเปลี่ยนข้อมูลระหว่างกันบนข้อมูลที่ใช้ร่วมกัน เราต้องมั่นใจว่าแต่ละทาสก์จะมีการเข้าถึงข้อมูลร่วมกันโดยหลีกเลี่ยงการแย่งชิงข้อมูล โดยวิธีการโดยทั่วไปที่จะเข้าถึงข้อมูลร่วมกันมีดังนี้

- ไม่ให้ใช้อินเทอร์รัพท์ (disable interrupts)
- ทำเทสต์และเซตโอเปอร์เรชัน (performing test-and-set operations)
- ไม่ให้มีการใช้การจัดลำดับ (disabling scheduling)
- ใช้เซมาฟอว์ (using semaphores)

3.2.4 เซมาฟอว์ (Semaphores)

เป็นโปรโตคอลสำหรับมัลติทาสก์กิ้งเคอร์เนล (multitasking kernels) โดยส่วนใหญ่ มีจุดประสงค์เพื่อ

- ควบคุมการเข้าถึงทรัพยากรร่วม
- ส่งสัญญาณของเหตุการณ์ที่เกิดขึ้น
- ยอมให้ 2 ทาสก์ทำการซิงค์ (sync) งานกันได้

เซมาฟอว์เป็นกุญแจที่จะทำให้โค้ดของเราเรียงลำดับการทำงาน ถ้าเซมาฟอว์กำลังถูกใช้งานอยู่ทาสก์ที่เรียกเข้ามาจะถูกพักไว้ชั่วคราวจนกระทั่งเซมาฟอว์ถูกปล่อยจากเจ้าของปัจจุบัน โดยแบ่งเซมาฟอว์ได้เป็น 2 ชนิด คือ binary semaphore และ counting semaphore ดังเช่นที่ชื่อได้แสดงไว้ binary semaphore สามารถมีค่าได้เพียงแค่ 0 หรือ 1 ส่วน counting semaphore สามารถมีค่าได้ตั้งแต่ 0 ถึง 255 , 65535 , 4294967295 แล้วแต่โครงสร้างของแต่ละเซมาฟอว์ที่เขียนโดยใช้ 8, 16, 32 บิต แต่ขนาดจริงๆ จะขึ้นอยู่กับที่เคอร์เนลที่ใช้ เมื่อมีการนำเซมาฟอว์มาใช้งาน ตัวระบบเองต้องมีการเก็บเส้นทางของทาสก์ที่ทำการรอเพื่อให้เซมาฟอว์ดำเนินการได้

โดยปกติแล้ว มีเพียงแค่ 3 โอเปอร์เรชันที่ถูกเตรียมไว้ใช้ในเซมาฟอว์ INITIALIZE (หรือเรียกว่า CREATE), WAIT (หรือเรียกว่า PEND), SIGNAL (หรือเรียกว่า POST) สำหรับค่าเริ่มต้นของเซมาฟอว์ต้องถูกเตรียมในช่วงที่เซมาฟอว์ถูก initialized และทาสก์ที่รออยู่ของเซมาฟอว์จะต้องถูกทำให้ว่าง

ทาสก์ที่ต้องการเซมาฟอว์จะเรียก WAIT แล้วถ้าเซมาฟอว์นั้นสามารถใช้งานได้ (ค่ามีค่าเกิน 0) ค่าของเซมาฟอว์จะถูกลดค่าลงและทาสก์จะสามารถทำงานได้ต่อไป แต่ถ้าค่าของเซมาฟอว์มีค่าเป็น 0 แล้วทาสก์มีการเรียก WAIT จะถูกทำการถูกนำไปไว้ใน waiting list โดยเคอร์เนล ส่วนใหญ่จะยอมให้กำหนด timeout เพื่อกำหนดเวลาที่รอเซมาฟอว์ โดยจะมีการเรียกทาสก์ขึ้นมารันและแสดงข้อผิดพลาดกลับไปยังผู้ที่เรียกทาสก์นั้น

ทาสก์ที่ปล่อย เซมาฟอว์ออกมาจะไปเรียก SIGNAL ถ้าไม่มีทาสก์อื่นรออยู่ใน waiting list ค่าของเซมาฟอว์ก็จะเพิ่มขึ้น แต่ถ้ามีทาสก์เหลืออยู่ในลิสต์จะทำการเรียกทาสก์ในลิสต์ขึ้นมารันโดยไม่มีที่เพิ่มค่าเซมาฟอว์ สำหรับเงื่อนไขในการหยิบทาสก์ขึ้นมาทำงานนั้น ขึ้นอยู่กับเคอร์เนลดังเช่น

- ทาสก์ที่มีระดับความสำคัญสูงสุดที่รออยู่
- ทาสก์แรกที่สามารถรอในลิสต์ (First In First Out หรือ FIFO)

3.2.5 การสื่อสารระหว่างทาสก์ (Intertask Communication)

บางครั้งก็เป็นความจำเป็นของทาสก์หรือไอเอสอาร์ (ISR) ที่จะติดต่อข้อมูลกับทาสก์อื่น ๆ ข้อมูลที่แลกเปลี่ยนเรียกการสื่อสารระหว่างทาสก์ โดยข้อมูลที่ถูกใช้ในการสื่อสารระหว่างทาสก์ทำได้ 2 ทาง คือ ผ่านตัวแปรกลาง หรือ ส่งข้อความ

เมื่อใช้การผ่านตัวแปรกลาง แต่ละทาสก์หรือไอเอสอาร์ต้องแน่ใจว่ามีการเข้าถึงข้อมูลเพียงทีละหนึ่งทาสก์เท่านั้น ถ้าไอเอสอาร์ได้ถูกเรียกแล้วมีทางเดียวที่จะเข้าถึงข้อมูลได้อย่างปลอดภัยคือ การไม่ให้ใช้อินเทอร์รัพท์ (disabled interrupt) แต่ถ้ามี 2 ทาสก์ที่ขอใช้งานตัวแปรร่วมตัวเดียวกันสามารถทำได้โดยไม่ให้อินเทอร์รัพท์หรือเรียกใช้เซมาฟอร์

ทาสก์สามารถติดต่อกับไอเอสอาร์ได้เพียงเดียวเท่านั้นคือผ่านตัวแปรกลาง ซึ่งโดยที่จริงแล้วทาสก์ไม่จำเป็นต้องกลัวว่าไอเอสอาร์จะไปเปลี่ยนค่าในตัวแปรกลาง จึงไม่มีความจำเป็นที่ไอเอสอาร์จะต้องส่งสัญญาณไปยังทาสก์ โดยใช้เซทาฟอร์หรือโพลล์ (poll) เพื่อให้เหมาะกับสถานการณ์เช่นนี้ คุณควรเลือกใช้แมสเสจเมล์บ็อกซ์ (message mailbox)

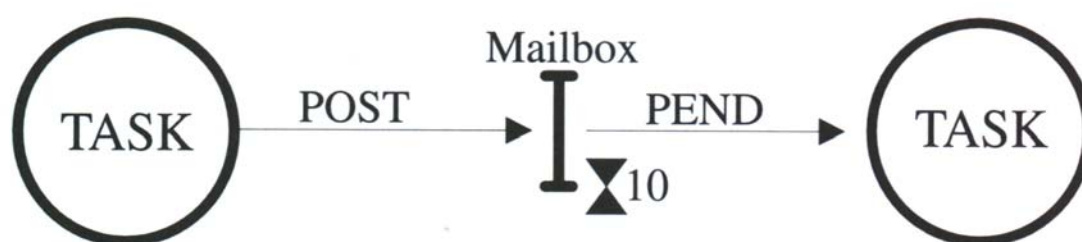
3.2.6 แมสเสจเมล์บ็อกซ์

ข้อความสามารถถูกส่งไปยังทาสก์ผ่าน “ kernel services “ คือแมสเสจเมล์บ็อกซ์หรือเรียกว่า “ Message exchange “ โดยเก็บผ่านตัวแปรพอยเตอร์ โดยทาสก์หรือไอเอสอาร์สามารถฝากแมสเสจลงในเมล์บ็อกซ์และ 1 หรือ หลายทาสก์สามารถรับแมสเสจผ่านบริการที่เคอร์เนลเตรียมไว้ โดยการส่งและรับข้อความจะเป็นการส่งข้อมูลที่พอยเตอร์นั้น ๆ ซ้ำอยู่

Waiting list ที่สัมพันธ์แต่ละเมล์บ็อกซ์ ในกรณีที่มี 1 หรือ หลาย ๆ ทาสก์ต้องการรับข้อความผ่าน เมล์บ็อกซ์ โดยทาสก์ที่ต้องการข้อความจากเมล์บ็อกซ์ที่ว่าง จะถูกทำให้พักการทำงานและถูกทำให้รออยู่ใน Waiting list จนกระทั่งมีข้อความเข้ามาในเมล์บ็อกซ์ โดยเคอร์เนลอนุญาตให้รอได้จนถึง timeout ถ้าข้อความยังไม่ถึงก่อนเวลาที่กำหนดไว้ทาสก์ที่รอรับข้อความจะถูกทำให้ ready และทำงาน โดยส่ง error code กลับไป เมื่อข้อความได้นำออกจากเมล์บ็อกซ์ทาสก์ที่มีระดับความสำคัญสูงที่สุดจะทำการรับข้อความ (แบบ priority based) หรือทาสก์แรกที่เข้ามารอเป็นผู้รับข้อความ (แบบ FIFO based)

เคอร์เนลได้เตรียมบริการสำหรับเมล์บ็อกซ์ดังนี้

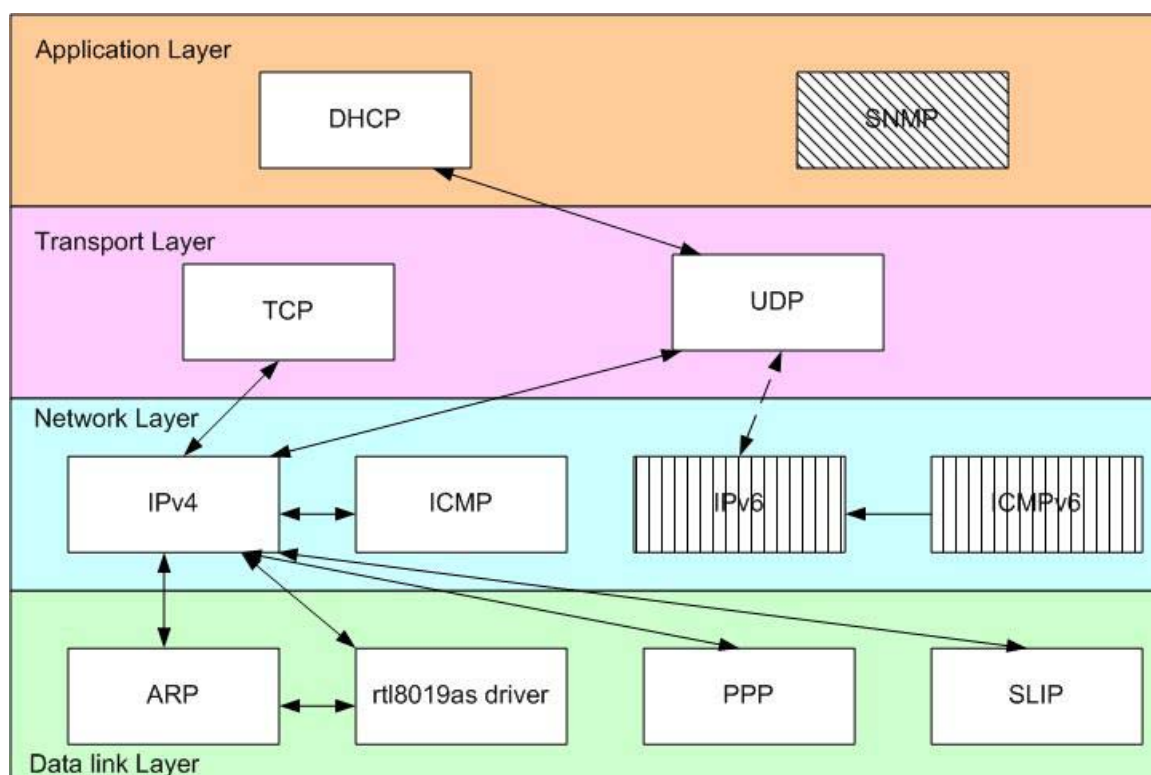
- กำหนดค่าเริ่มต้นของเนื้อหาในเมล์บ็อกซ์ในตอนแรกเริ่มเมล์บ็อกซ์สามารถกำหนดให้มีหรือไม่ มีแมสเสจได้
- ฝากแมสเสจลงในเมล์บ็อกซ์ (POST)
- รอรับแมสเสจที่ได้ทำการฝากลงในเมล์บ็อกซ์ (PEND)
- รับแมสเสจจากเมล์บ็อกซ์ ถ้ามีข้อความอยู่ แต่จะไม่พักการทำงานของผู้ที่เรียกถ้าเมล์บ็อกซ์ ว่างเปล่า (ACCEPT) ถ้าเมล์บ็อกซ์มีข้อความอยู่ ข้อความจะถูกนำออกจากเมล์บ็อกซ์ และส่งโค้ดกลับจะถูกนำไปใช้ในการเตือนผู้ที่เรียกเกี่ยวกับผลลัพธ์ของการเรียก



รูปที่ 3-2 แมสเสจเซมัลบ็อกซ์

3.3 ไลต์เวททีซีพี/ไอพีแอสตีก (Light weight protocol stack : lwIP)

ไลต์เวททีซีพี/ไอพีแอสตีกเป็นการอิมพลีเมนต์ทีซีพี/ไอพีแอสตีก ไลต์เวททีซีพี/ไอพีแอสตีกนั้นมีเป้าหมายในการลดขนาดการใช้หน่วยความจำลงและขนาดของโค้ด ทำให้ไลต์เวททีซีพี/ไอพีแอสตีกนั้นเหมาะกับที่จะใช้ในไมโครคอนโทรลเลอร์ที่มีขนาดเล็กมีทรัพยากรจำกัดอย่างเช่น ระบบฝังตัว เป็นต้น โดยสามารถที่จะลดพารามิเตอร์และการใช้หน่วยความจำได้ ไลต์เวททีซีพี/ไอพีแอสตีกใช้วิธีการของ API ซึ่งทำให้ไม่จำเป็นต้องคัดลอกข้อมูล โดยแสดงโครงสร้างของทีซีพี/ไอพีแอสตีกได้ดังภาพ



รูปที่ 3-3 โครงสร้างของไลต์เวททีซีพี/ไอพีแอสตีก

3.3.1 โพรเซสโมเดล (Process Model)

แบ่งออกเป็น 2 แบบด้วยกัน

1. ทำให้แต่ละโปรโตคอลรันแบบสแตนด์อโลนโพรเซส (stand alone process) หมายความว่าแต่ละโปรโตคอลจะแยกกันทำงานเดี่ยวๆ ด้วยโมเดลแบบนี้จะเข้มงวดในการบังคับโปรโตคอลเลเยอร์ (Strict Protocol layering) และจุดของการติดต่อสื่อสารระหว่างโปรโตคอลจะเข้มงวดในการกำหนด ซึ่งมีข้อดีคือ โปรโตคอลสามารถถูกเพิ่มเข้าไปในระบบในระหว่างรันไทม์ (run time) ได้ ทำให้การทำความเข้าใจโค้ดและการดีบั๊กเป็นไปได้ง่ายขึ้น แต่ก็มีข้อเสียคือ การที่จะทำให้เป็นโปรโตคอลที่เป็นแบบเข้มงวดในการบังคับโปรโตคอลเลเยอร์นั้นต้องมีการทำคอนเท็กซ์สวิตชิง (context switching) เช่น ถ้ามีทิสพีเช็กเมนต์เข้ามาจะมีการทำ 3 คอนเท็กซ์สวิตชิง จากคิววอร์เควอร์ไปยังเน็ตเวิร์คอินเทอร์เฟซ จากไอพีโพรเซสไปยังทิสพีโพรเซส แล้วจากทิสพีโพรเซสไปยังแอปพลิเคชันโพรเซส เป็นต้น

2. ทำให้การสื่อสารระหว่างโปรโตคอลนั้นอยู่ข้างๆกับเคอร์เนลของโอเอส (เคอร์เนลอิมพลีเมนต์เทชัน) โดยแอปพลิเคชันโพรเซสจะสื่อสารกับโพรเซสผ่านทางซิสเต็มคอล (System calls) ใช้เทคนิคของการข้ามชั้นโปรโตคอล (crossing the protocol layering) เพราะโปรโตคอลนั้นไม่ได้ถูกแบ่งออกเป็นชั้นอย่างชัดเจนเหมือนแบบที่ 1

ไลต์เวททิสพี/ไอพีแอสต์กั้นใช้โพรเซสโมเดลแบบให้โปรโตคอลนั้นอยู่ข้างๆกับซิงเกิลโพรเซส ซึ่งแยกออกจากเคอร์เนลของระบบปฏิบัติการ โดยที่แอปพลิเคชันโปรแกรมสามารถที่จะอยู่ข้างๆไลต์เวททิสพี/ไอพีแอสต์กั้นโพรเซสได้หรือเป็นโพรเซสแยกต่างหากก็ได้ การสื่อสารระหว่างทิสพี/ไอพีแอสต์กั้นและแอปพลิเคชันโปรแกรมจะกระทำโดยการใช้ฟังก์ชันในการเรียก สำหรับกรณีที่แอปพลิเคชันโปรแกรมนั้นแบ่งปันโพรเซสกับไลต์เวททิสพี/ไอพีแอสต์กั้น หรือใช้ abstract API ไลต์เวททิสพี/ไอพีแอสต์กั้นจะใช้โพรเซสในส่วนของผู้ใช้ (user space process) แทนที่จะใช้เคอร์เนลของระบบปฏิบัติการ ซึ่งมีข้อดีก็คือ สามารถพอร์ตข้ามระบบปฏิบัติการที่ต่างกันได้ง่าย ถ้าออกแบบให้ใช้งานกับระบบปฏิบัติการขนาดเล็ก มันจะไม่รองรับ swapping out process กับหน่วยความจำเสมือน (virtual memory) ทำให้ไม่เกิดความล่าช้า (delay) จากการที่ต้องรอ I/O (disk) ถ้าแค่บางส่วนของไลต์เวททิสพี/ไอพีแอสต์กั้นโพรเซสนั้นถูก swap หรือ page out ไปยังดิสก์นั้นไม่ใช่ปัญหา แต่ก็มีข้อเสียคือ มีการรอ scheduling quantum ก่อนที่จะมีโอกาสที่จะขอรับบริการได้

3.3.2 อิมูเลชันเลเยอร์ของระบบปฏิบัติการ (The Operating System Emulation Layer)

ในการที่จะทำให้ไลต์เวททิสพี/ไอพีแอสต์กั้นสามารถพอร์ตกับระบบปฏิบัติการต่างๆได้นั้น ระบบปฏิบัติการจะไม่ได้เรียกใช้ฟังก์ชันเฉพาะและดาต้าสตรักเจอร์โดยตรงจากโค้ด และเมื่อจำเป็นที่จะต้องเรียกใช้ก็จะใช้อิมูเลชันเลเยอร์ขึ้นมาแทน ซึ่งเป็นยูนิฟอร์มอินเทอร์เฟซ (uniform interface) ไปยังโอเอสเซอร์วิส เช่น ไทม์เมอร์ โพรเซสซิงโครไนซ์เซชัน กระบวนการแมสเสสพาสซิง เป็นต้น การที่จะพอร์ตไลต์เวททิสพี/ไอพีแอสต์กั้นกับระบบปฏิบัติการอื่นนั้นก็เพียงแก้ไขในส่วนนี้ให้เหมาะสมกับระบบปฏิบัติการที่จะใช้

อิมูเลชันเลเยอร์ของระบบปฏิบัตินั้นมีฟังก์ชันเกี่ยวกับไทม์เมอร์ที่ถูกใช้งานโดยทิสพีเป็นแบบอันชีอตไทม์เมอร์ (one-shot timer) ที่อย่างน้อย 200 มิลลิวินาทีที่จะเรียกกรีจิสเตอร์ฟังก์ชันขึ้นมา (เมื่อเกิด time-out)

สำหรับกระบวนการโพสซึ่งโครโนชันนั้นใช้เซมาฟอร์ ถ้าระบบปฏิบัติการนั้นไม่มีการทำเซมาฟอร์ก็จะจำลองแบบอื่นๆ ขึ้นมาใช้งานได้ เช่น เงื่อนไขของตัวแปร หรือล็อก (Lock)

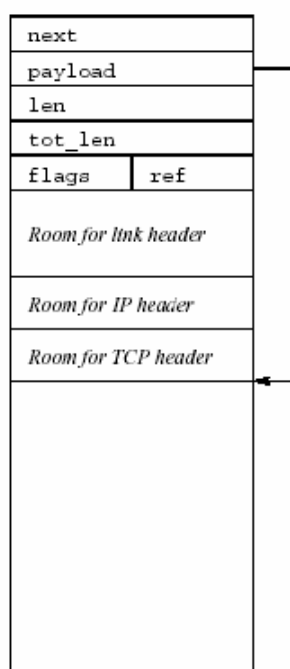
เมสเสจพาสซึ่งนั้นที่การกระทำผ่านการใช้ abstraction called mailboxes โดยที่เมล์บ็อกซ์นั้นมี 2 โอเปอเรชันคือ post กับ fetch โดยที่การ post นั้นจะไม่บล็อกโพสและเมสเสจที่ถูก post ในเมล์บ็อกซ์จะถูกจัดคิว โดยอีเมลชันเลขอร์ของระบบปฏิบัติการจนกว่าจะมีโพสอื่นมา fetch มันออกไป

3.3.3 การจัดการบัฟเฟอร์และหน่วยความจำ (Buffer and Memory management)

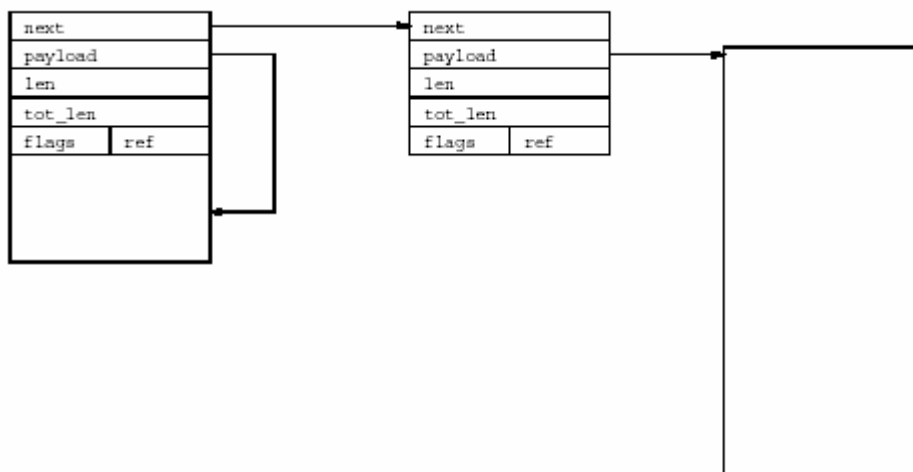
3.3.3.1 แพ็กเก็ตบัฟเฟอร์ (Packet buffers-pbufs)

pbuf ใช้เป็นตัวแทนของแพ็กเก็ตภายในไลทเวทชีฟ/ไอพีสแต็ค และถูกออกแบบมาสำหรับความต้องการพิเศษที่จะใช้สแต็คน้อยๆ ซึ่งคล้ายกันกับ mbuf ใน BSD implementations โดยที่โครงสร้างของ pbuf จะสนับสนุนการกำหนดพื้นที่แบบไดนามิกส์ เพื่อเก็บแพ็กเก็ตคอนเท้นท์และสำหรับแพ็กเก็ตดาต้าที่อยู่ข้าง static memory pbuf สามารถเชื่อมเข้าด้วยกันแบบลิงค์ลิสต์ได้เรียกว่า pbuf chain ดังนั้นบางทีแพ็กเก็ตถูกแบ่งเป็นหลายๆ ส่วน

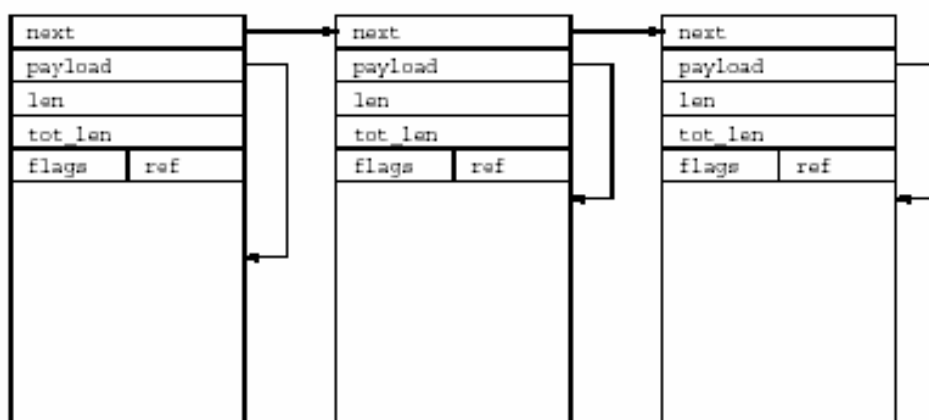
โดยที่ pbuf นั้นมี 3 แบบคือ pbuf_ram, pbuf_rom, pbuf_pool ดังนี้



รูปที่ 3-4 pbuf_ram



รูปที่ 3-5 *pbuf_ram* เชื่อมต่อเข้ากับ *pbuf_rom* ที่มีข้อมูลอยู่ที่ *external memory*



รูปที่ 3-6 *chained pbuf_pool* จาก *pbuf_pool*

pbuf_pool นั้นใช้โดยเน็ตเวิร์คไดรเวอร์ โดยที่โอเพอร์เรชันในการกำหนดพื้นที่ของ single *pbuf* นั้นเร็วและเหมาะสมกับที่จะใช้กับอินเทอร์พอร์ทแฮนด์เคิลเลอร์ ส่วน *pbuf_rom* นั้นจะใช้เมื่อโปรแกรมประยุกต์ส่งข้อมูลที่อยู่ในหน่วยความจำที่จัดการโดยโปรแกรมประยุกต์ ข้อมูลเหล่านั้นจะไม่ถูกเปลี่ยนแปลงหลังจาก *pbuf* ได้เก็บไว้ในที่ซีพี/ไอพีแอสตีกและเมื่อข้อมูลอยู่ในรอม (*pbuf_rom*) แล้ว

pbuf ของ *pbuf_ram* นั้นจะถูกใช้เมื่อโปรแกรมประยุกต์ส่งข้อมูลเป็นไดนามิกส์ *Pbuf system* จะจองหน่วยความจำ ไม่เพียงแต่ข้อมูลของโปรแกรมประยุกต์ แต่จะมี Header จองเตรียมไว้ล่วงหน้า และในแบบที่แย่มากที่สุด (worse case) ขนาดของ Header จะถูกติดตั้งในเวลาที่ยากลำบากด้วย

โดย *pbuf* ที่เข้ามาจะเป็น *pbuf_pool* และ *pbuf* ที่ออกไปจะเป็น *pbuf_ram* หรือ *pbuf_rom* โครงสร้างของ *pbuf* นั้นประกอบด้วย 2 pointers, 2 length fields, Flag field, และตัวนับ *next* คือพอยเตอร์ไปยัง *pbuf* ถัดไปในกรณีที่มีการ chain “Payload pointer” เป็นตัวชี้จุดเริ่มของข้อมูล “Len field” เก็บความยาวของข้อมูลใน *pbuf* “Tot_len field” เก็บผลรวมของความยาวปัจจุบันและความยาวทั้งหมดใน chain ในอีกความหมายหนึ่งคือ *tot_len field* เป็นผลรวมความยาวของฟังก์ชันและค่าของ *tot_len* ในการตาม *pbuf* ใน *pbuf chain* “Flag

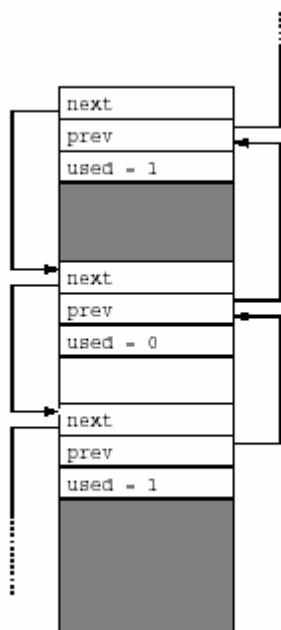
field” ใช้แสดงชนิดของ pbuf และ “ref field” จะประกอบด้วย reference count โดยที่ “next” และ “payload field” นั้นเป็น pointer ที่มีมาตั้งแต่แรก และขนาดจะแปรตามสถาปัตยกรรมของโปรเซสเซอร์ที่ใช้ “two len field” เป็น 16 บิตอันไหนอินทิจอร์และ flag กับ ref เป็น 4 บิต ขนาดทั้งหมดของ pbuf ขึ้นอยู่กับขนาดของพอยเตอร์ในสถาปัตยกรรมของโปรเซสเซอร์ สำหรับในสถาปัตยกรรม 32 บิต และ 4 ไบต์จะไลน์เมนต์ ขนาดทั้งหมด 16 ไบต์และบนสถาปัตยกรรมที่เป็น 16 บิตพอยเตอร์และ 1 ไบต์จะมีขนาดรวม 9 ไบต์

3.3.3.2 การจัดการกับหน่วยความจำ (Memory management)

ใช้ memory manager ทำหน้าที่กำหนดและยกเลิกกำหนดพื้นที่ที่ติดกับของหน่วยความจำและสามารถลดขนาดที่ได้กำหนดไว้ก่อนหน้านี้ได้ memory manager ใช้บางส่วนของหน่วยความจำทั้งหมดที่ได้รับในระบบ โอเปอร์เรชันของโปรแกรมอื่นๆจะไม่เข้ามาจนได้ ถ้าเน็ตเวิร์คซิสเต็มใช้หน่วยความจำทั้งหมด

Memory manager จะติดตามการกำหนดหน่วยความจำ โดยการเอาโครงสร้างขนาดเล็กไปแปะไว้กับด้านบนของแต่ละบล็อกของหน่วยความจำที่กำหนดไว้ โครงสร้างนี้มี 2 พอยเตอร์ชี้ไปยังตัวต่อไปและตัวก่อนหน้า และยังใช้ flag ในการบอกว่าบล็อกนั้นถูกกำหนดไปแล้วหรือยัง

หน่วยความจำจะถูกกำหนดโดยการบล็อกที่กำหนดไว้ที่ยังไม่ถูกใช้ที่มีขนาดใหญ่เพียงพอตามที่ขอมา โดยใช้อัลกอริทึม “first fit” เมื่อมีการยกเลิกกำหนดพื้นที่แล้ว use flag ก็จะถูกกำหนดเป็น 0 เพื่อป้องกันการเฟรกเมนต์ use flag ของก่อนหน้าและถัดไป ตัวบล็อกจะถูกตรวจสอบก่อน ถ้ามีอันใดก็ตามที่มีสถานะเป็น unused อยู่ บล็อกก็จะถูกรวมเป็นบล็อกของ unused ที่ใหญ่ขึ้น



รูปที่ 3-7 โครงสร้างของการกำหนดหน่วยความจำ

3.3.4 โพรโทคอลที่สนับสนุน

เนื่องจากไลต์เวททีซีพี/ไอพีแอสต์กั้นมีการสนับสนุนการทำงานของโปรโตคอลไม่ครบทุกอย่าง จึงสามารถจำแนกได้ดังตารางต่อไปนี้

Protocol	Feature support	หมายเหตุ
ARP	ARP request	ARP packet จะต้องเป็น local broadcast packet บน local link เท่านั้น
	ARP reply	
	RFC 3220 4.6 IP Mobility Support for IPv4	
SLIP		จะต้องมี serial I/O เพราะเป็นการส่ง IP packet บน Dial-up line module
PPP	Link Control Protocol	PPP IP Control Protocol
	Network Control Protocols (NCPs)	
	Authentication and Phase control	
	Network Challenge Handshake Authentication Protocol (CHAP)	
	Network Password Authentication Protocol (PAP)	
	MD5	
IPv4	Fixed IP address	ทำ local network binding โดยใช้ wildcard
	Class A	
	Class B	
	Class C	
	Class D	
	Broadcast address	
	Loop back address	
	IP fragmentation	
	IP reassemble	
	IP checksum	
ICMP	Network Unreachable	
	Host Unreachable	
	Protocol Unreachable	
	Port Unreachable	

	Fragmentation needed but DF set	ต้องอนุญาตให้มีการทำ IP forward ต้องอนุญาตให้มีการทำ IP forward
	Source route failed	
	Time to live exceeded in transit	
	Fragment reassembly time exceeded	
IPv6	IPv6 header	
	IP checksum	
ICMPv6	Network Unreachable	ต้องอนุญาตให้มีการทำ IP forward ต้องอนุญาตให้มีการทำ IP forward
	Host Unreachable	
	Protocol Unreachable	
	Port Unreachable	
	Fragmentation needed and DF set	
	Time to live exceeded in transit	
	Fragment reassembly time exceeded	
TCP	Multiple TCP connections	
	TCP options	
	Variable TCP Maximum Segment Size	
	Round Trip Time Estimation	
	TCP Flow Control	
	Sliding TCP Window	
	TCP Congestion Control	
	Out-of-Sequence TCP data	
	TCP urgent data	
	Data buffered for retransmit	
UDP	UDP lookup	Incomplete
DHCP	Client mode	
SNMP		Define function แต่ไม่มี code ในการทำงาน

ตารางที่ 3-1 โปรโตคอลที่สนับสนุน

3.3.5 โครงสร้างข้อมูลที่เกี่ยวข้อง

3.3.5.1 eth_addr

เป็นโครงสร้างของแอดเดรส

3.3.5.2 eth_hdr

เป็นโครงสร้างของ header ของเออาร์พีแพ็กเก็ต แสดงโครงสร้างได้ดังรูป



รูปที่ 3-8 โครงสร้างของ *eth_hdr*

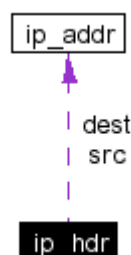
ซึ่งประกอบด้วย source, destination และ type

3.3.5.3 ip_addr

เป็นโครงสร้างของไอพีแอดเดรสในรูปแบบของเน็ตเวิร์คไบต์ออร์เดอร์ (Network byte order) ที่มีค่าตัวฟิลต์เป็น `u32_t addr`

3.3.5.4 ip_hdr

เป็นเฮดเดอร์ของไอพีแพ็กเก็ต โดยมีโครงสร้างดังนี้



รูปที่ 3-9 โครงสร้างของ *ip_hdr*

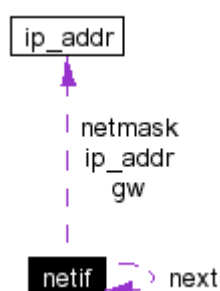
ซึ่งประกอบด้วย

<code>_v_hl_tos</code>	version, header length, type of service
<code>len</code>	total length
<code>_id</code>	identification
<code>_offset</code>	fragment offset
<code>_ttl_proto</code>	time to live, protocol
<code>_chksum</code>	checksum
<code>src</code>	source IP address
<code>dest</code>	destination IP address

3.3.5.5 netif

เป็นตัวแทนเน็ตเวิร์คอินเทอร์เฟซ โดยจะเป็นตัวที่เชื่อมโยงระหว่างเสต็กกับเน็ตเวิร์คอินเทอร์เฟซ โดยที่เน็ตเวิร์คอินเทอร์เฟซอาจจะเป็นดีไวซ์ไครเวอร์ไปยังฟิสิคัลเน็ตเวิร์คฮาร์ดแวร์ก็ได้ หรืออาจจะเป็นเวรปเปอร่าไปยังอินเทอร์เฟซอื่น เช่น /dev/tun หรือ /dev/tap (UNIX/Linux)

โดยที่เน็ตเวิร์คอินเทอร์เฟซจะถูกเก็บไว้ในนโกลบอลลิงค์ลิส (Global linked list) ซึ่งเชื่อมโยงโดยพอยเตอร์ดังโครงสร้างต่อไปนี้



รูปที่ 3-10 โครงสร้างของ *netif*

ซึ่งประกอบด้วย

next	อินเทอร์เฟซถัดไปใน global connected list
num	จำนวนของอินเทอร์เฟซใน global connected list
ip_addr	โลคอลไอพีแอดเดรสในแบบเน็ตเวิร์คไบต์ออร์เดอร์
netmask	เน็ตมาสก์ในแบบเน็ตเวิร์คไบต์ออร์เดอร์
gw	ไอพีแอดเดรสของเกตเวย์ในแบบเน็ตเวิร์คไบต์ออร์เดอร์
hwaddr	ฮาร์ดแวร์แอดเดรสของดีไวซ์
name	ชื่อของแต่ละเน็ตเวิร์คอินเทอร์เฟซ
state	ระบุถึงสถานะของอุปกรณ์โดยดีไวซ์ไครเวอร์

3.3.5.6 pbuf

โครงสร้างของเซกเตอร์ของแพ็กเก็ตบัฟเฟอร์ ออกแบบมาเพื่อให้เหมาะกับเสต็กขนาดเล็ก โดยเฉพาะซึ่งคล้ายกับ mbuf ของ BSD ซึ่งรองรับทั้งการกำหนดพื้นที่หน่วยความจำแบบไดนามิกซ์และแบบคงที่สามารถเชื่อมต่อกันใน 1 ลิสต์ได้ เรียกว่า pbuf chain ดังที่ได้กล่าวไว้แล้วข้างต้น



รูปที่ 3-11 โครงสร้างของ *pbuf*

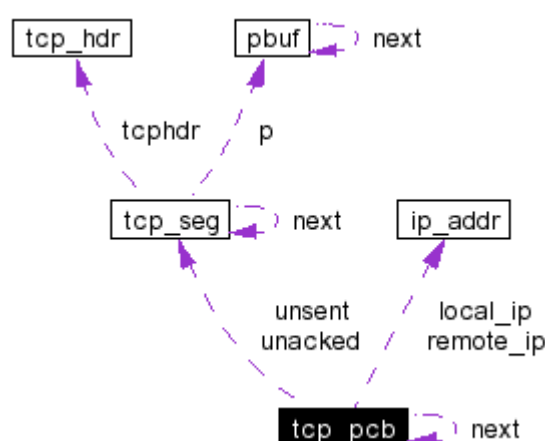
3.3.5.7 tcp_hdr

โครงสร้างของเฮดเดอร์ของทีซีพีแพ็กเก็ต ซึ่งประกอบด้วย

src	ต้นทาง
dest	ปลายทาง
seqno	เลขลำดับ (sequence number)
ackno	หมายเลขตอบรับ (Acknowledge number)
_offset_flags	ออฟเซตและเฟล็ก
wnd	วินด์โดว์
chksum	checksum
urgp	urgent pointer

3.3.5.8 tcp_pcb

โครงสร้างของโปรโตคอลคอนโทรลบล็อกสำหรับทีซีพีคอนเนกชัน ซึ่งประกอบด้วยข้อมูลสำหรับทีซีพีคอนเนกชัน และเก็บอยู่บนลิงค์ลิส นอกจากนี้ยังมีพอยเตอร์ไปยัง PCB ถัดไปใน global linked list ของ TCP PCBs ด้วย สามารถแสดงโครงสร้างได้ดังนี้



รูปที่ 3-12 โครงสร้างของ `tcp_pcb`

ซึ่งประกอบด้วย

next	พอยเตอร์ไปยัง PCB ถัดไปใน global linked list ของ TCP PCBs
callback_arg	สำหรับ callback ฟังก์ชัน
local_ip	โลคอลไอพีแอดเดรส
local port	โลคอลพอร์ต
remote_ip	รีโมทไอพีแอดเดรส
remote_port	รีโมทพอร์ต
state	TCP state
rtime	รีทรานสมิตชันไทม์เมอร์

mss	maximum segment size
rto	retransmission time-out
nrtx	จำนวนในการรีทรานสมิต
snd_buf	ขนาดของบัฟเฟอร์ที่มีอยู่สำหรับใช้ส่งเป็นไบต์
snd_queuelen	ขนาดของบัฟเฟอร์ที่มีอยู่สำหรับใช้ส่งเป็นทึชีพีเชกเมนต์
unsent	คิวของเชกเมนต์ที่ยังไม่ได้ส่งโดยเรียงตามเลขลำดับ
unacked	ที่ส่งแล้วแต่ยังไม่ได้รับ ACK กลับเรียงตามเลขลำดับ
rcv_nxt	เลขลำดับถัดไปที่คาดหวังว่าจะได้
rcv_wnd	วินโดว์ของผู้รับ
tmr	ไทม์เมอร์
polltmr	ไทม์เมอร์สำหรับการพูล
pollinterval	ช่วงเวลาในการพูล
flags	ค่าเฟรกต่างๆ
rttest	ค่าประมาณของราวทริปไทม์
rtseq	มีการจับเวลาของเลขลำดับนั้น
sa	ค่าประมาณของราวทริปไทม์
sv	ค่าประมาณของราวทริปไทม์
lastack	ค่าเลขลำดับของ ACK สูงสุดที่ได้รับ
dupacks	ค่าเลขลำดับของ ACK สูงสุดที่ได้รับ
cwnd	congestion avoidance/control
ssthresh	congestion avoidance/control
snd_nxt	เลขลำดับถัดไปที่จะส่ง
snd_max	ค่าเลขลำดับสูงสุดที่ส่ง
snd_wnd	วินโดว์ของผู้ส่ง
snd_wl1	เลขลำดับและเลขตอบรับของวินโดว์สุดท้ายที่ได้รับการแก้ค่า
snd_wl2	เลขลำดับและเลขตอบรับของวินโดว์สุดท้ายที่ได้รับการแก้ค่า
snd_lbb	เลขลำดับของไบต์ถัดไปที่จะถูกบัฟเฟอร์

3.3.5.9 tcp_pcb_listen

โครงสร้างของสถานะของทึชีพีในโหมด LISTEN และ TIME_WAIT เพราะมีความจำเป็นที่จะต้องเก็บสถานะน้อยกว่าสถานะอื่นๆเลยสามารถใช้ PCB ที่มีขนาดเล็กกว่าปกติในการเก็บได้



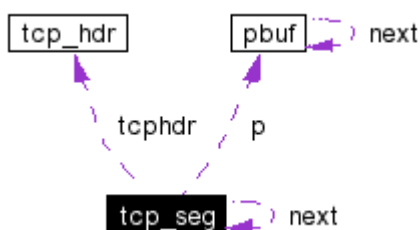
รูปที่ 3-13 โครงสร้างของ *tcp_pcb_listen*

ซึ่งประกอบด้วย

next	พอยเตอร์ไปยัง PCB ถัดไปใน global linked list ของ TCP PCBs
callback_arg	สำหรับ callback ฟังก์ชัน
local_ip	โลคอลไอพีแอดเดรส
local_port	โลคอลพอร์ต

3.3.5.10 tcp_seg

เป็นโครงสร้างของทีซีพีเซกเมนต์เมื่ออยู่ในคิว



รูปที่ 3-14 โครงสร้างของ *tcp_seg*

ซึ่งประกอบด้วย

next	ใช้เมื่อมีการเอาเซกเมนต์ไปใส่คิว
p	บัฟเฟอร์ที่ประกอบด้วยข้อมูลและเฮดเดอร์ของทีซีพี
dataptr	พอยเตอร์ที่ชี้ไปยังทีซีพีดาต้าภายใน pbuf
len	ความยาวของทีซีพีในเซกเมนต์นี้
tcphdr	เฮดเดอร์ของทีซีพี

3.3.5.11 udp_hdr

โครงสร้างของเฮดเดอร์ของยูดีพี ซึ่งประกอบด้วย

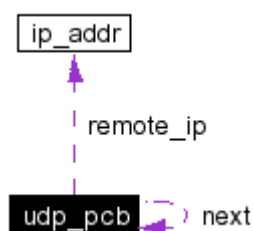
src	source
dest	destination
len	ความยาวของยูดีพี

chksum

checksum

3.3.5.12 udp_pcb

โครงสร้างของโปรโตคอลคอนโทรลบล็อกของยูดีพีคอนเนคชัน โดยจะเก็บข้อมูลเกี่ยวกับ 1 ยูดีพีคอนเนคชันใดๆ ซึ่งจะเก็บบนลิงค์ลิสต์ที่ตรงกับยูดีพีดาต้าแกรมที่เข้ามา นอกจากนี้ยังมีพอยน์เตอร์ที่ชี้ไปยัง PCB ถัดไปที่อยู่บน global linked list ของ UDP PCBs ซึ่งสามารถแสดงโครงสร้างได้ดังนี้



รูปที่ 3-15 โครงสร้างของ *udp_pcb*

ซึ่งประกอบด้วย

next	พอยน์เตอร์ที่ชี้ไปยัง PCB ถัดไปใน global linked list ของ UDP PCBs
remote_ip	ไอพีแอดเดรสของปลายทางของยูดีพีเซสชัน
local_port	หมายเลขพอร์ตของต้นทางของยูดีพีเซสชัน
remote_port	หมายเลขพอร์ตของปลายทางของยูดีพีเซสชัน
flags	ค่าเฟร็กที่ระบุถึงข้อกำหนดของ checksum ของยูดีพีนั้นต้องใช้เซสชันนี้
chksum_len	ใช้ในการบอกขนาด checksum ในโหมด UDP lite